

PySpark Customer Behavior Scoring Project

Executive Summary

This project is a production-like MVP pipeline for customer behavior scoring on the Synerise event log dataset. The current end-to-end path is:

```
Synerise event logs
-> PySpark processing
-> user-level features
-> purchase propensity label
-> Spark ML baseline model
-> batch score table
-> FastAPI/SQLite lookup API
-> manual lightweight prediction path
-> sanitized demo/testing artifacts
```

The MVP is not a full production deployment. Its main purpose is to practice the production workflow: scalable data processing, leakage-safe feature and label construction, baseline modeling, batch scoring, API serving, documentation, and privacy controls. The current milestone is Phase 7: API Serving & Demo Interface. The lookup API and manual prediction endpoint are implemented locally; demo UI, load/stress testing, external deployment, authentication, and production monitoring remain pending.

1. Problem Statement

Customer behavior scoring means converting customer activity records into user-level signals that can support prediction. Event logs can be aggregated into features such as visit frequency, cart activity, purchase behavior, search activity, recency, and interaction trends.

The project builds a customer behavior scoring pipeline from event logs. The pipeline uses PySpark to inspect, process, and aggregate behavioral data into user-level features. The MVP has implemented a purchase propensity prediction path through batch scoring and a local API serving layer.

The current outcome is a practical local pipeline that can process customer behavior data, create user-level features, train a simple model, generate batch prediction results, serve lookup scores by user/client id, and compute manual feature-input predictions using lightweight exported Logistic Regression metadata.

2. Why This Project Matters

Behavior scoring is useful because it turns raw interaction history into prediction-ready information. It can support customer churn prediction, propensity scoring, campaign targeting, personalized user analysis, and a user-level prediction API.

From an engineering perspective, the project is also useful because it practices scalable data processing on event logs, including schema inspection, joins, aggregations, feature creation, batch scoring, and local API serving.

3. Project Direction

Chosen direction: Customer behavior scoring from event logs.

Current MVP direction:

```
Raw event logs
-> PySpark processing
-> EDA
-> preprocessing
-> feature engineering
-> model training
-> batch prediction table
-> API lookup
-> manual feature-input prediction
```

The first version stays MVP-focused before adding advanced recommendation, embedding, challenge-reproduction methods, external deployment, or production hardening.

Current Status by Phase

Phase	Status	Main tool	Output	Review point
Raw inspection / EDA	Done	PySpark, Parquet metadata	Sanitized schema, row count, null, and timestamp artifacts	Confirm data coverage and core entities
Business target selection EDA	Done	PySpark aggregate analysis	Purchase propensity selected with 30-day target window	Confirm target is business-appropriate

Phase	Status	Main tool	Output	Review point
Preprocessing	Done	PySpark DataFrame transforms	Clean event/product Parquet tables	Review duplicate handling and deferred <code>page_visit</code>
Feature engineering	Done	PySpark aggregations and joins	36 user-level features	Review feature sufficiency and null strategy
Label construction	Done	PySpark time-window joins	Purchase propensity labels and training dataset	Review cutoff/window boundary policy
Baseline modeling	Done	Spark ML Logistic Regression	Baseline model and aggregate metrics	Review threshold and false-positive trade-off
Batch scoring	Done	Spark ML batch transform	Local score table with stable serving schema	Review score distribution and refresh cadence
API lookup / manual prediction	Implemented	FastAPI, SQLite, lightweight LR metadata	<code>GET /scores/{client_id}</code> and <code>POST /predict</code>	Review local API contract and demo behavior
Demo UI	Not done yet	Streamlit or lightweight frontend, TBD	No UI artifact yet	Choose demo approach
Load/stress testing	Not done yet	k6, Locust, or simple script, TBD	No performance report yet	Define API/cache test target
External deployment/auth/Redis	Out of MVP or pending	External infrastructure, TBD	Not implemented	Future hardening only

4. Dataset Understanding

Dataset selected: Synerise RecSys 2025.

Raw files currently found in `data/raw/`:

File	Size	Inferred format	Spark-readable	Verified schema	Row count status
<code>data/raw/synerise_dataset/add_to_cart.parquet</code>	100.54 MB	Parquet	Yes	<code>client_id: long, timestamp: string, sku: long</code>	7,541,117 rows
<code>data/raw/synerise_dataset/page_visit.parquet</code>	1.87 GB	Parquet	Yes	<code>client_id: long, timestamp: string, url: long</code>	199,451,980 rows
<code>data/raw/synerise_dataset/product_buy.parquet</code>	30.05 MB	Parquet	Yes	<code>client_id: long, timestamp: string, sku: long</code>	2,318,502 rows
<code>data/raw/synerise_dataset/product_properties.parquet</code>	64.24 MB	Parquet	Yes	<code>sku: long, category: long, price: long, name: string</code>	1,534,050 rows
<code>data/raw/synerise_dataset/remove_from_cart.parquet</code>	34.84 MB	Parquet	Yes	<code>client_id: long, timestamp: string, sku: long</code>	2,688,894 rows
<code>data/raw/synerise_dataset/search_query.parquet</code>	335.74 MB	Parquet	Yes	<code>client_id: long, timestamp: string, query: string</code>	13,223,769 rows

What is known so far:

- The dataset is currently available as six Spark-readable Parquet files.
- The main event tables use `client_id` and `timestamp`.
- Product-related event tables use `sku`.
- Product metadata is available in `product_properties.parquet` with `sku`, `category`, `price`, and `name`.
- The EDA job was rerun in full-count mode, and all six Spark-readable Parquet tables were counted successfully.
- The metadata artifact is intentionally sanitized. It records table-level schema and size information only, without absolute local paths or row-level data.

What has been verified beyond raw inspection:

- Duplicate behavior was profiled in preprocessing artifacts and remains a policy review point.
- Purchase propensity labels were constructed with a leakage-safe cutoff and target window.
- The 30-day purchase propensity target was carried through feature engineering, modeling, batch scoring, and local API serving.

Potential entities to look for during inspection:

- client/user
- event
- item/product
- timestamp
- target/label if available

Initial schemas, exact row counts, timestamp parsing, tracked null counts, feature tables, labels, baseline model, batch scores, and local serving artifacts are available. Remaining work is focused on demo UI, load/stress testing, threshold review, optional tuning, and deployment hardening.

5. Why PySpark Is Used

The dataset may be processable with local tools, but PySpark is used here to practice scalable data processing patterns such as reading event logs, schema inspection, aggregation, joins, window-based features, and batch scoring.

Spark concepts planned for this project include:

- reading event logs
- schema inspection
- DataFrame transformations
- joins
- aggregations
- window-based features
- batch scoring

Tooling and Design Rationale

Tool / approach	Used in phase	Why used	Alternative	Why not alternative / trade-off
PySpark	EDA, preprocessing, feature engineering, label construction, modeling, batch scoring	Handles large event logs better than Pandas, supports distributed-style DataFrame processing even in local mode, and fits the internship goal of scalable data workflow practice	Pandas	Pandas is simpler for small data, but less appropriate for large joins, aggregations, and production-like event processing. Spark has more overhead and debugging complexity.
Parquet	Raw, processed, feature, label, training, and score tables	Columnar, schema-aware, efficient with Spark, and better suited to analytical workloads than row text formats	CSV	CSV is easy to inspect manually, but slower, less type-safe, and less efficient for Spark workloads.
Spark ML Logistic Regression	Baseline modeling	Simple, explainable, built into Spark ML, and produces probability scores usable for ranking and TopK targeting	RandomForest, GBT, XGBoost, deep models	More complex models may capture nonlinear behavior, but they add complexity. Logistic Regression is a faster, clearer MVP baseline.
Median imputation	Model input preparation	Keeps rows with missing numeric values and is more robust than mean for skewed features	Drop rows, mean imputation, constant fill, missing indicators	Median imputation can hide missingness signal. Missing indicators or more explicit null handling are future improvements.
Class weights	Baseline modeling	The positive label rate is low, so class weights reduce the risk of a model that mostly predicts the negative class	Resampling, threshold tuning only, anomaly-style modeling	Class weights improve recall but can increase false positives. Threshold selection remains a review point.
Batch scoring	Serving preparation	Keeps API fast and simple by precomputing scores offline and avoiding Spark/model inference per request	Realtime Spark inference	Realtime inference can be fresher but is heavier, slower, and harder to operate for this MVP. Batch scores can become stale until refresh.
SQLite	Local lookup serving store	Lightweight local store for MVP API lookup and tests without external infrastructure	Postgres, Redis, cloud database	SQLite is not a production high-concurrency multi-instance serving database, but it is appropriate for local demo and API contract validation.
FastAPI	API layer	Lightweight Python framework with clear request/response models, health/metadata endpoints, and easy testability	Flask, Django, Node/Fastify	FastAPI is suitable for this MVP, but production still needs auth, deployment, monitoring, rate limiting, and access control.

Tool/ approach metadata	Used in phase Manual feature- input prediction	Allows API to compute Logistic Regression sigmoid directly from feature order, imputation values, coefficients, intercept, and threshold without Spark in the request path	Alternative: Spark ML model inside API	Spark in request handlers would be heavy and operationally fragile for a local API. Metadata export requires careful versioning and validation.
500000-row local serving export	API/cache demo preparation	Provides a larger local lookup dataset for API/cache assignment testing while keeping row-level data ignored by git	Full production database export or tiny sample only	Larger local export is useful for demo testing, but it is still local data and not a production serving store.
Markdown report	Mentor-facing documentation	Keeps decisions, metrics, artifacts, privacy notes, and limitations in one reviewable source of truth	Many small documents	A single report can become long, so it needs clear sections and review-oriented summaries.

6. Planned Pipeline

Stage	Objective	Input	Process	Output	What to review
Problem statement	Define the prediction-oriented project goal	Internship direction and dataset context	Frame event logs as user-level scoring inputs	Clear behavior scoring objective	Whether the project direction is useful and explainable
Dataset understanding	Identify available data and possible entities	Files under <code>data/raw/</code>	Inventory files, inspect archive members, identify formats	Raw data summary and dataset notes	Whether required entities appear to exist
Data ingestion	Make Spark-readable input tables available	Parquet files under <code>data/raw/synerise_dataset/</code>	Read Parquet with Spark and inspect schemas	Spark DataFrames or staged raw tables	Schema, row counts, data types, and table relationships
EDA	Understand data quality and behavior patterns	Spark-readable raw tables	Count rows, inspect nulls, timestamp ranges, event distributions, skew	EDA statistics and observations	Data quality issues and useful behavior signals
Preprocessing	Clean and standardize source tables	Raw Spark DataFrames	Normalize timestamps, handle duplicates/nulls, filter invalid records	Clean intermediate tables	Cleaning rules and possible information loss
Feature engineering	Build user-level predictive signals	Clean event and metadata tables	Aggregate events by user, create recency/frequency/activity features	User-level feature table	Feature meaning, leakage risk, and null defaults
Label definition	Define the first supervised target	Event timelines and feature table	Choose churn or propensity logic, separate feature and label windows	Training dataset with label	Whether the label is valid and mentor-approved
Modeling	Train a simple baseline model	Labeled training table	Encode features, split train/validation, train baseline model	Model artifact, metrics, and result table	Model validity, metrics, and tuning priorities
Batch scoring	Generate prediction outputs for serving	Feature table and trained model	Score users in batch and store latest predictions	Batch prediction table	Prediction schema and refresh logic
API serving and demo interface	Return scores by user/client id and support manual feature-input review	Batch prediction table and exported lightweight model metadata	Implement lookup endpoint, manual feature scoring, and simple demo UI	API endpoint and mentor demo interface	Request/response format, error cases, and demo clarity
Experiment tracking and tuning	Compare model variants offline	Training dataset and model configuration	Train Spark model variants, log sanitized aggregate metrics, and compare thresholds/parameters	Experiment artifacts and comparison table	Which parameter set and threshold should be selected

Stage	Objective	Input	Process	Output	What to review
Early stress test and demo packaging	Check API behavior under repeated requests and prepare final review	Running API and demo UI	Run basic repeated request tests and document final architecture	Latency notes, final report, and demo flow	Whether performance and documentation are acceptable for demo

7. Project Decisions

Decision 1: Problem Direction

Chosen: Customer behavior scoring from event logs.

Reason: This matches the dataset structure and the internship requirement to practice PySpark preprocessing, modeling, and API prediction serving.

Alternatives considered:

- Pure recommendation
- Universal behavioral embedding
- Churn/propensity scoring

Implemented choice: Business target selection EDA was used to choose the MVP target. The implemented target is purchase propensity with a 30-day target window.

Decision 2: Dataset

Chosen: Synerise RecSys 2025.

Reason: The dataset contains customer behavior event logs, timestamps, product metadata, and enough size/complexity to justify PySpark practice.

Limitation: The dataset is not Spark-only. It may be processed with other local tools, but PySpark is used to practice scalable data processing patterns.

Decision 3: Processing Approach

Chosen: Use PySpark local mode first.

Reason: Local mode is easier to develop and explain, while still using Spark concepts such as schema inspection, DataFrame operations, joins, aggregations, and window-based features.

Decision 4: MVP Scope

MVP:

- Raw data inspection
- EDA
- User-level feature table
- Purchase propensity model
- Batch prediction table
- Simple API lookup
- Manual feature-input prediction from lightweight model metadata

Out of scope for MVP:

- Deep learning embeddings
- Full RecSys challenge reproduction
- Real-time Spark serving
- Distributed cluster deployment
- External production database deployment
- Authentication, access control, and production monitoring

Decision 5: Data Packaging and Ingestion

Chosen: Inspect the available raw files first, then use the Spark-readable Parquet tables as ingestion inputs.

Reason: Spark reads structured files such as Parquet, CSV, and JSON. The current dataset files are available as Parquet tables, so they can be inspected directly in local Spark mode. If the dataset is later provided as an archive, archive handling should happen before Spark table ingestion.

8. Detailed Reporting Template for Future Stages

This section defines how future project updates should be documented for mentor review.

8.1 EDA Reporting Template

For each EDA result, document:

- Question
- Input table/columns
- PySpark operation used
- Result
- Interpretation
- Follow-up decision

Example EDA questions:

- How many events are in each table?
- How many unique clients exist?
- What is the timestamp range?
- Which event type dominates?
- Are there missing values?
- Are there skewed users/items?

8.2 Feature Engineering Reporting Template

For each feature group, document:

- Feature group name
- Source table
- Input columns
- Transformation logic
- Output columns
- Why the feature may help prediction
- Null/default handling

Example format:

Feature group	Source	Input columns	Output columns	Logic	Reason
TBD	TBD	TBD	TBD	TBD	TBD

8.3 Modeling Reporting Template

For each model experiment, document:

- Experiment name
- Prediction task
- Label definition
- Training input
- Feature columns
- Categorical encoding
- Numerical preprocessing
- Model selected
- Hyperparameters
- Train/validation split
- Metrics
- Results
- Observations
- Possible fine-tuning
- Mentor review questions

Example format:

Item	Detail
Task	Churn prediction
Label	TBD
Input table	TBD
Features	TBD
Model	TBD
Parameters	TBD
Metrics	TBD
Result	TBD
Fine-tuning options	TBD

8.4 API Reporting Template

For API implementation later, document:

- Endpoint
- Input parameters

- Prediction source table
- Output schema
- Example response
- Error cases
- Load/stress test result
- Cache strategy if used

EDA Milestone Summary

Objective: Understand the verified raw tables, core columns, timestamp quality, missing values, and table relationships before preprocessing or feature engineering.

Input tables:

- add_to_cart
- page_visit
- product_buy
- product_properties
- remove_from_cart
- search_query

Process: The EDA job reads the raw Parquet tables with PySpark in local mode, computes sanitized table-level metrics, and writes structured outputs without row-level samples. The job was rerun in full-count mode.

EDA approach: EDA was used not only to look at the data, but also to decide whether the project is feasible as a Spark workflow and what later phases should build. The job inspects schemas, row counts, approximate distinct counts, null counts for core columns, timestamp parse success, timestamp min/max values, product price summaries, and sanitized table/column artifacts. These checks inform the target definition, feature plan, privacy boundary, and runtime strategy.

Safe mode vs full count: The first inspection used safe-mode behavior and size thresholds to avoid accidentally triggering expensive Spark actions on large files. Safe mode did not mean the data was unreadable. After confirming schemas and file readability, the full-count EDA job was executed intentionally to collect exact table-level statistics. This is a cautious workflow for large local data: inspect cheaply first, then run heavier actions deliberately.

Output artifacts:

- artifacts/eda/eda_summary.json
- artifacts/eda/table_overview.csv
- artifacts/eda/event_table_overview.csv
- artifacts/eda/product_table_overview.csv
- artifacts/eda/column_overview.csv

The EDA artifacts were split into compact general overview and schema-specific views to avoid sparse columns across heterogeneous input tables.

Why table-level and column-level summaries: Table-level summaries explain whether each dataset is readable and large enough for modeling. Column-level summaries explain whether core identifiers, timestamps, event keys, and null patterns are suitable for feature construction. This avoids relying on raw samples, which would be less privacy-safe and less representative for large event logs.

Key findings:

- All six raw tables are Spark-readable.
- Core event tables contain client_id and timestamp.
- Product event tables contain sku.
- product_properties contains sku, category, price, and name.
- Null counts are zero for tracked core columns in this EDA run.
- Timestamp parsing succeeded for all non-null timestamp values in the event tables.
- All six Spark-readable Parquet tables were counted successfully in full-count mode.
- The earlier deferred row counts for page_visit and search_query were caused by the safe-mode strategy, not by Spark readability limitations.

Compact table overview:

Table	Row count	Main columns	Role in pipeline
page_visit	199,451,980	client_id, timestamp, url	High-volume browsing behavior
search_query	13,223,769	client_id, timestamp, query	Search behavior signal
add_to_cart	7,541,117	client_id, timestamp, sku	Cart intent behavior
remove_from_cart	2,688,894	client_id, timestamp, sku	Cart reversal behavior
product_buy	2,318,502	client_id, timestamp, sku	Purchase behavior and target selection input
product_properties	1,534,050	sku, category, price, name	Product metadata for category/price features

Finding: Event Tables Are Spark-Readable

Input: All six raw Parquet tables.

Process: PySpark read.parquet was applied to each expected table path.

Result: All six tables returned spark_readable = true in artifacts/eda/table_overview.csv.

Interpretation: The project can proceed using Spark DataFrames directly from the raw Parquet files.

Next decision: Confirm whether preprocessing should operate directly from these raw tables or write cleaned intermediate outputs.

Finding: Core Event Columns Are Available

Input: Schemas from `add_to_cart`, `page_visit`, `product_buy`, `remove_from_cart`, and `search_query`.

Process: The EDA job recorded schema field names and data types for each table.

Result: Event tables include `client_id` and `timestamp`. Product-related event tables include `sku`; page visits include `url`; search behavior includes `query`.

Interpretation: `client_id` is a strong candidate user key, `timestamp` is the event-time field, and `sku` supports product-level joins for cart and purchase behavior.

Next decision: Review whether `client_id`, `timestamp`, and `sku` should be accepted as the core preprocessing columns.

Finding: Full-Count Row Counts Are Available

Input: `row_count` and `row_count_status` from `artifacts/eda/table_overview.csv`.

Process: The EDA job was rerun with `--full-count`, which deliberately computes exact row counts for every readable table.

Result: All six row counts are available: `page_visit` (199,451,980), `search_query` (13,223,769), `add_to_cart` (7,541,117), `remove_from_cart` (2,688,894), `product_buy` (2,318,502), and `product_properties` (1,534,050).

Interpretation: The largest table is `page_visit`, followed by `search_query`. Earlier deferred counts were safe-mode behavior, not a Spark readability limitation.

Next decision: Use these volumes to plan preprocessing actions carefully, especially for `page_visit`.

Finding: Timestamp Parsing Is Clean

Input: `timestamp` columns in the event tables.

Process: The EDA job parsed timestamps with PySpark `to_timestamp`, then counted parse successes and failures.

Result: All event tables with `timestamp` had zero null timestamps and zero timestamp parse failures. The observed timestamp range spans from 2022-06-23 to 2022-12-08.

Interpretation: Timestamp fields appear suitable for time-window analysis, churn-window definition, and recency/frequency features.

Next decision: Choose the feature window and target window strategy before label definition.

Finding: Product Metadata Is Useful for Features

Input: `product_properties` columns `sku`, `category`, `price`, and `name`.

Process: The EDA job computed row count, approximate distinct counts, null counts, and safe numeric price summary statistics without outputting product names.

Result: `product_properties` has 1,534,050 rows, approximately 1,369,522 distinct SKUs, approximately 6,347 categories, zero nulls in tracked metadata columns, and price values from 0 to 99 with average 47.8629 and median 47.0.

Interpretation: Product metadata can support category and price-based feature groups after preprocessing.

Next decision: Decide whether product metadata should be joined before aggregation or after event-level cleaning.

Open questions:

- Are approximate distinct client counts sufficient for planning, or should exact distinct counts be computed for selected tables?
- Should duplicate and skew checks be added before or during preprocessing?
- Should purchase propensity remain the MVP target after preprocessing validation?
- What time window should define features and labels?

Mentor review points:

- Confirm whether `client_id`, `timestamp`, and `sku` are acceptable core columns.
- Confirm whether full-count EDA is enough to begin preprocessing.
- Review whether product category and price should be used in the MVP feature set.
- Review whether the first modeling task should be purchase propensity.

Next decision before feature engineering: Move to preprocessing only after the core columns, timestamp strategy, and first target direction are reviewed.

Business Target Selection EDA

Objective: Compare multiple business scoring targets that fit the available e-commerce behavior logs before selecting the first MVP target.

The goal of this EDA step is not to force a churn prediction task, but to compare multiple business scoring targets that fit the available e-commerce behavior logs. The first MVP target should be selected based on data coverage, label balance, implementation complexity, and business actionability.

Candidate targets compared:

- Purchase propensity: which active clients are likely to purchase in the target window.
- Cart conversion: among clients who showed cart intent, who will convert to purchase in the target window.
- Purchase-based churn: among clients who purchased before, who will not purchase again in the target window.

Input: Primary inputs were `product_buy` and `add_to_cart`. `page_visit` and `search_query` were skipped by default for this decision step because the default

target comparison can be computed from purchase and cart behavior without scanning the largest table.

Process: The job evaluated 14-day, 30-day, and 45-day target windows using aggregate-only PySpark operations. It computed eligible clients, positive clients, negative clients, positive rate, implementation complexity, business value, and simple recommendation ranks. It did not write final labels or client-level outputs.

Why this approach: Target selection was handled as aggregate EDA before label construction so the team could compare business usefulness, class balance, and implementation complexity before committing to a supervised task. The alternative was to pick churn or purchase propensity by intuition, but that would make the label choice harder to defend in mentor review.

Target window comparison:

Target days	History start	Cutoff date	Target end	History days
14	2022-06-23	2022-11-25	2022-12-08	155
30	2022-06-23	2022-11-09	2022-12-08	139
45	2022-06-23	2022-10-25	2022-12-08	124

Business target comparison:

Target	Window	Eligible clients	Positive clients	Negative clients	Positive rate	Complexity	Business value	Rank
Purchase propensity	14	2,418,073	57,754	2,360,319	0.023884	medium	high	2
Purchase propensity	30	2,149,796	93,614	2,056,182	0.043546	medium	high	1
Purchase propensity	45	1,925,028	112,154	1,812,874	0.058261	medium	high	1
Cart conversion	14	2,114,526	47,724	2,066,802	0.022570	low to medium	high	4
Cart conversion	30	1,872,709	75,415	1,797,294	0.040271	low to medium	high	4
Cart conversion	45	1,672,381	89,320	1,583,061	0.053409	low to medium	high	3
Purchase-based churn	14	826,947	778,744	48,203	0.941710	low	medium	5
Purchase-based churn	30	739,219	661,550	77,669	0.894931	low	medium	5
Purchase-based churn	45	668,496	576,651	91,845	0.862609	low	medium	5

Business meaning:

- Purchase propensity is broad and suitable for ranking active users by likelihood to buy.
- Cart conversion is action-oriented for cart recovery, but its eligible cohort is narrower than purchase propensity.
- Purchase-based churn is clear and simple, but the positive class means churn and is highly dominant, which can make interpretation and baseline modeling less balanced.

Implementation complexity:

- Purchase propensity: medium, because it requires an active-user cohort and future purchase label.
- Cart conversion: low to medium, because the cohort is clearly defined by add-to-cart behavior.
- Purchase-based churn: low, because it uses only purchase history, but it has narrower business scope and stronger imbalance.

Recommended MVP target: Based on the comparison, the recommended MVP target is purchase propensity with a 30-day target window because it has broad data coverage, high business actionability, and a better overall recommendation rank than the churn alternatives. The 45-day purchase propensity option has the same overall rank and a higher positive rate, so it remains a strong alternative for review. This decision remains provisional until preprocessing validates duplicate handling, time-window boundaries, and leakage-safe feature construction.

Provisional label definition: Eligible clients have at least 1 `add_to_cart` or `product_buy` event before `cutoff_date` . Positive clients have at least 1 purchase from `cutoff_date` to `target_end` . Negative clients have no purchase in that target window.

Why other targets are alternatives, not discarded:

- Cart conversion remains useful if the MVP should focus specifically on cart recovery.
- Purchase-based churn remains useful for retention analysis, but it is narrower and more imbalanced in this dataset.
- The 45-day purchase propensity window remains a strong alternative if the team prefers a less sparse positive class.

Leakage prevention rule: Features must only use events before `cutoff_date` . Labels must only use purchase events from `cutoff_date` to `target_end` . No target-window behavior should be used in feature calculations.

Open questions before preprocessing/feature engineering:

- Should the MVP use the recommended 30-day purchase propensity target or the 45-day purchase propensity alternative?
- Should the eligible cohort include only add-to-cart and purchase behavior, or later include search/page activity?
- Should duplicate and skew checks be added in preprocessing or targeted follow-up EDA?
- Should class imbalance handling be planned during modeling?

Review / limitation: The selected 30-day purchase propensity target is now implemented through modeling and serving, but target choice remains a business review point. The 45-day window and cart conversion target remain reasonable alternatives if mentor feedback prioritizes a different action window or narrower campaign use case.

Preprocessing Milestone Summary

Objective: Prepare clean, standardized intermediate tables for later feature engineering and label construction while keeping the current MVP target as purchase propensity.

Input tables:

- `add_to_cart`
- `remove_from_cart`
- `product_buy`
- `search_query`
- `product_properties`

Deferred input:

- `page_visit` was deferred because it is the largest table and is not required for the initial purchase propensity preprocessing path.

Process: The preprocessing job reads raw Parquet tables with PySpark, parses event timestamps, creates `event_ts` and `event_date` , adds an `event_type` , validates required columns, checks aggregate duplicate rates, and writes cleaned intermediate Parquet outputs with Spark. Product metadata is cleaned into modeling-safe columns only: `sku` , `category` , and `price` .

Why this approach: Preprocessing uses Spark DataFrames because the event tables are large and later phases depend on consistent schemas, parsed timestamps, and repeatable Parquet outputs. Product names and raw query text are excluded from shared outputs to reduce privacy exposure. The alternative was to clean data ad hoc inside feature engineering, but a separate preprocessing step makes validation and review easier.

Output data:

- `data/processed/events/add_to_cart/`
- `data/processed/events/remove_from_cart/`
- `data/processed/events/product_buy/`
- `data/processed/events/search_query/`
- `data/processed/product_properties_clean/`

These processed data outputs are local pipeline outputs and are ignored by git.

Output artifacts:

- `artifacts/preprocessing/preprocessing_summary.json`
- `artifacts/preprocessing/table_validation.csv`
- `artifacts/preprocessing/duplicate_check_summary.csv`
- `artifacts/preprocessing/product_metadata_validation.csv`
- `artifacts/preprocessing/preprocessing_notes.md`

Validation results:

Table	Input rows	Valid rows	Invalid rows	Timestamp parse failures	Write status
<code>add_to_cart</code>	7,541,117	7,541,117	0	0	success
<code>remove_from_cart</code>	2,688,894	2,688,894	0	0	success
<code>product_buy</code>	2,318,502	2,318,502	0	0	success
<code>search_query</code>	13,223,769	13,223,769	0	0	success
<code>product_properties</code>	1,534,050	1,534,050	0	not applicable	success

Duplicate handling:

Table	Duplicate key	Duplicate extra rows	Duplicate extra row rate
<code>add_to_cart</code>	<code>client_id</code> , <code>event_ts</code> , <code>sku</code>	161,493	0.021415

remove_from_cart	client_id, event_ts, sku	153,588	0.057119
Table	Duplicate key	Duplicate extra	Duplicate extra row
product_buy	client_id, event_ts, sku	rows 380,903	rate 0.164288
search_query	client_id, event_ts	787,135	0.059524

Interpretation: The duplicate checks are aggregate diagnostics only. No duplicate row samples are persisted. Duplicate handling should be reviewed before feature engineering chooses whether repeated identical events should count as repeated behavior or be collapsed.

Product metadata handling:

product_properties has 1,534,050 input rows and 1,534,050 distinct SKUs in the preprocessing artifact. No duplicated SKU rows were detected in this run. The clean product metadata output excludes product names and keeps only sku, category, and price. The documented deterministic fallback rule is to group by sku and keep the most frequent category/price pair, with ties sorted by category and price.

Leakage-safe target setup:

The pipeline configuration records the provisional MVP target as purchase propensity with a 30-day target window. Preprocessing does not create final labels. Later feature construction must use only events before the cutoff date, while labels must use purchase behavior only in the target window.

Open questions:

- Should duplicate event rows be counted as repeated activity or deduplicated before feature aggregation?
- Should search_query remain included for Phase 3 features even though raw query text is not retained in shared artifacts?
- Should page_visit stay deferred for the MVP or be included after resource review?
- Are sku, category, and price sufficient product metadata columns for the first feature set?

Review / limitation: The duplicate policy is still a review point. Current features count processed rows as behavior, which may be correct if repeated events indicate intensity, but may overstate behavior if duplicates are system artifacts. page_visit remains deferred for MVP scope and runtime control.

Next decision before feature engineering: Approve the cleaned event/product tables and duplicate handling strategy before moving to Phase 3 feature engineering.

Feature Engineering Milestone Summary

Objective: Create leakage-safe user-level feature tables for the provisional purchase propensity task.

Input processed tables:

- data/processed/events/add_to_cart/
- data/processed/events/remove_from_cart/
- data/processed/events/product_buy/
- data/processed/events/search_query/
- data/processed/product_properties_clean/

Deferred input:

- page_visit remains deferred and is not used in the Phase 3 feature table.

Leakage rule: Features use events before the configured cutoff date, 2022-11-09. Target-window events from 2022-11-09 through 2022-12-08 are not used in feature calculations. This phase does not create final labels.

Feature groups generated:

- Activity count features for add-to-cart, remove-from-cart, purchase, and search activity.
- Recency features using days since the last event of each processed event type.
- Distinct interaction features such as distinct SKU counts and distinct search days.
- Ratio features such as buy-to-cart, remove-to-cart, and search-to-cart ratios.
- Product metadata features using category and price from cleaned product metadata.
- Windowed 30-day, 60-day, and 90-day lookback count features.
- Eligible cohort indicator for the later purchase propensity label construction step.

Why this approach: The feature set emphasizes interpretable recency, frequency, product, and search activity signals because they are explainable and can be built with Spark aggregations. More advanced alternatives such as embeddings, sequence models, or session modeling could capture richer behavior but would increase complexity before the baseline serving path is validated.

Output feature table:

- data/processed/features/user_behavior_features/

Output artifacts:

- artifacts/features/feature_summary.json
- artifacts/features/feature_catalog.csv
- artifacts/features/feature_validation.csv
- artifacts/features/feature_notes.md

Validation results:

- Feature rows: 2,810,342
- Eligible cohort count: 2,149,796
- Eligible cohort rate: 0.764959
- Feature count: 36
- Search features included: True
- page_visit included: False
- Label-like columns detected: none

Selected feature validation examples:

Feature	Null rate	Min	Max	Average
add_to_cart_count	0.000000	0	1,560	1.998546
product_buy_count	0.000000	0	644	0.626121
search_query_count	0.000000	0	3,264	3.602982
active_days_count	0.000000	1	125	1.629863
is_eligible_purchase_propensity	0.000000	0	1	0.764959

Null and fill strategy: Count-style features are filled with 0. Recency features remain null when a client has no event of that type. Ratio features remain null when the denominator is 0.

Duplicate handling assumption: Features are computed from processed rows as-is. Phase 2 duplicate diagnostics remain part of the review context, and the final duplicate policy is deferred for review before label construction or modeling.

Interpretation: The generated feature table covers all clients observed in the processed event inputs before the cutoff. The eligible cohort indicator identifies clients with add-to-cart or purchase activity before cutoff, but it is not a final label. This keeps Phase 3 scoped to feature creation only.

Review questions after feature engineering:

- Should duplicate event rows count as repeated behavior, or should specific duplicate keys be collapsed before modeling?
- Are the current 36 features sufficient for a baseline purchase propensity model?
- Should `page_visit` remain deferred for the MVP?
- Should missing recency and ratio values stay null or be filled during modeling preparation?

Review / limitation: The current 36 features are sufficient for a baseline MVP, but they are not a final feature set. Missingness itself may carry useful signal, and future iterations can add missing indicators, page-visit features, or richer temporal/session features.

Next decision before modeling: Review the generated labels and confirm whether the current feature table is sufficient for baseline modeling.

Label Construction Milestone Summary

Objective: Create a supervised purchase propensity label and a training-ready dataset for baseline modeling.

Input feature table: `data/processed/features/user_behavior_features/`

Input event table: `data/processed/events/product_buy/`

Process: The label job selects eligible clients from the Phase 3 feature table, reads target-window purchase events, aggregates purchases to one binary label row per client, and joins labels back to the eligible feature rows to create a training-ready dataset.

Why this approach: The label uses a fixed cutoff date and future purchase window to keep historical features separated from target behavior. This makes the task explainable: predict whether an active client will buy in the next 30 days. The alternative was to use churn or cart conversion first, but purchase propensity has broader coverage and supports user ranking for campaigns.

Cutoff date: `2022-11-09`

Target window: 30 days, from `2022-11-09` through `2022-12-08` .

Boundary rule: `event_ts >= cutoff_date` and `event_ts < date_add(target_end, 1)` .

Eligible cohort: 2,149,796 clients with add-to-cart or purchase activity in the history window.

Label definition: Positive label means an eligible client has at least one `product_buy` event in the target window. Negative label means no `product_buy` event in the target window.

Output label table: `data/processed/labels/purchase_propensity_30d/`

Output training-ready dataset: `data/processed/training/purchase_propensity_30d/`

Result:

Metric	Value
Label row count	2,149,796
Positive count	93,614
Negative count	2,056,182
Positive rate	0.043546
Training dataset row count	2,149,796

Feature count used Metric	Value	36
------------------------------	-------	----

Validation results:

- Label row count equals the eligible cohort count.
 - Positive count matches the Phase 1.1 purchase propensity 30-day result.
 - Label values are binary 0 and 1 .
 - No null labels were found.
 - No duplicate client_id values were found in the label table.
 - No duplicate client_id values were found in the training-ready dataset.
 - The training-ready dataset contains no prediction or model output columns.
- Leakage checks: Features come from the Phase 3 feature table, while labels use only product_buy events in the target window. No target-window features were created, and no preexisting label-like columns were found in the feature table.
- Duplicate handling: Multiple target-window purchases for the same client are aggregated into one binary label row. The count of target-window purchases is retained in the local label table, while the training-ready dataset keeps the final binary label.
- Null strategy: Feature nulls are preserved from Phase 3. Model-stage imputation is deferred to baseline modeling.
- Interpretation: The 30-day purchase propensity label is now available for the eligible cohort. The positive rate is low but usable for a baseline classification task, and the generated validation artifacts support moving to a modeling review gate.
- Review / limitation: The label is suitable for the MVP baseline, but it uses one cutoff. A stronger production evaluation would test multiple time-based cutoffs. Upstream duplicate policy also remains important because duplicate events can affect feature values even when the label table has one row per client.
- Next step: The label has already been used for baseline modeling. Remaining review points are boundary policy, class balance, and whether additional time-based backtesting is needed.

Baseline Modeling Milestone Summary

Objective: Train and evaluate a simple Spark ML baseline model for the 30-day purchase propensity task.

Input training dataset: data/processed/training/purchase_propensity_30d/

Model type: Spark ML Logistic Regression.

Train/test split: Deterministic 80/20 random split with seed 42 .

Split result:

Split	Rows	Positive rate
Train	1,720,719	0.043488
Test	429,077	0.043778

- Feature preparation: Numeric feature columns are assembled with Spark ML. Non-feature columns such as client_id, label, target-window metadata, and prediction/model columns are excluded.
- Imputation strategy: Median imputation for 36 numeric model input columns.
- Class imbalance handling: Class weights are enabled because the Phase 4 positive rate is 0.043546.
- Why this approach: Spark ML Logistic Regression is used as the first model because it is fast, explainable, built into Spark, and produces probability scores that can be ranked for targeting. Median imputation keeps sparse user histories in the dataset. Class weights address the low positive rate, while keeping the training process simple enough for mentor review.

Evaluation metrics:

Metric	Value
ROC-AUC	0.840501
PR-AUC	0.254436

Confusion matrix at threshold 0.5 :

Metric	Count
True positives	13,637
False positives	89,245
True negatives	321,048
False negatives	5,147

Threshold metrics:

Metric	Value
Precision	0.132550
Recall	0.725990
F1	0.224171

TopK targeting metrics:

K	Users	Positives captured	Precision@K	Recall@K	Lift@K
1%	4,291	2,062	0.480541	0.109774	10.976839
5%	21,454	6,355	0.296215	0.338320	6.766350
10%	42,908	9,234	0.215205	0.491589	4.915851

Model output path: data/models/purchase_propensity_baseline/

Artifacts:

- artifacts/modeling/baseline_model_summary.json
- artifacts/modeling/baseline_metrics.csv
- artifacts/modeling/topk_metrics.csv
- artifacts/modeling/feature_processing_summary.csv
- artifacts/modeling/baseline_model_notes.md

Current result: The Phase 5 baseline modeling job completed successfully. The ROC-AUC indicates the baseline ranks positives well above random. PR-AUC is substantially higher than the test positive rate, which is more informative than accuracy for this imbalanced task. TopK lift is strongest at the top 1%, which supports a ranking-based targeting use case.

Limitations: A single random split is acceptable for the MVP baseline. A future improvement should evaluate time-based backtesting over multiple cutoffs. The threshold 0.5 favors recall but creates many false positives, so operating threshold selection should be reviewed before production scoring. TopK metrics are computed with Spark ordering plus limit() for each K slice and are kept as aggregate artifacts only.

Modeling interpretation for review: ROC-AUC measures ranking ability across thresholds. PR-AUC is more informative than accuracy because the positive class is sparse. The confusion matrix at threshold 0.5 shows the operating behavior of the current default threshold: recall is high, but false positives are also high. TopK metrics matter because a business campaign usually contacts only the highest-ranked users, not every user above a generic probability threshold. Threshold selection is therefore a product and mentor review point, not just a modeling detail.

Next step: The baseline model has already been used for batch scoring. Remaining review points are threshold choice, TopK targeting objective, and whether optional model variants should be compared later.

Batch Scoring Milestone Summary

Objective: Generate serving-ready batch prediction scores for eligible purchase propensity clients.

Input feature table: data/processed/features/user_behavior_features/

Input model path: data/models/purchase_propensity_baseline/

Scoring cohort: Clients with is_eligible_purchase_propensity = 1 from the Phase 3 feature table.

Model version: baseline_lr_v1

Output score table: data/processed/scoring/purchase_propensity_scores/

Score columns:

- client_id
- prediction_score
- prediction_label
- model_version
- scored_at

Stable scoring schema: The serving-facing score table keeps a stable schema because downstream API clients rely on consistent field names and types. prediction_score is the class-1 purchase propensity probability. prediction_label is the thresholded decision. model_version and scored_at provide traceability for refreshes and model comparisons. This schema helps avoid breaking changes when the serving layer or demo UI reads score records.

Process: The batch scoring job loads the Phase 5 Spark ML model, applies it to the eligible Phase 3 feature rows, extracts the class-1 probability as prediction_score, applies the configured threshold for prediction_label, and writes a local Spark Parquet score table.

Why this approach: Batch scoring is used before API serving so scores are computed offline and served through fast lookup. This avoids running Spark or model inference per request. The alternative is realtime model inference, but that would be heavier and less suitable for the local MVP. The trade-off is freshness: scores are only as current as the latest batch refresh.

Artifacts:

- artifacts/scoring/scoring_summary.json
- artifacts/scoring/score_distribution.csv
- artifacts/scoring/scoring_validation.csv
- artifacts/scoring/scoring_notes.md

Validation checks: The job validates score row count, eligible row count, duplicate client_id, score range, null scores, prediction label values, model version, score timestamp, model load status, score write status, and absence of labels or target-window metadata in the scoring output.

Result:

Metric	Value
Input feature rows	2,810,342
Eligible scoring rows	2,149,796
Score output rows	2,149,796
Predicted positive count	516,759
Predicted positive rate	0.240376
Minimum prediction score	0.000000
Maximum prediction score	1.000000
Average prediction score	0.350679

Score distribution:

Score bucket	Row count	Row rate
0.0-0.1	36,854	0.017143
0.1-0.2	669,430	0.311392
0.2-0.3	507,557	0.236095
0.3-0.4	170,841	0.079468
0.4-0.5	248,355	0.115525
0.5-0.6	216,975	0.100928
0.6-0.7	105,137	0.048906
0.7-0.8	77,653	0.036121
0.8-0.9	54,793	0.025488
0.9-1.0	62,201	0.028933

Validation result: All scoring validation checks passed. The score output row count equals the eligible cohort count, no duplicate client_id values were found, scores are non-null and within [0, 1], prediction labels are binary, model version and score timestamp are populated, and labels or target-window metadata are excluded from the output.

Leakage checks: Scoring input comes from the Phase 3 feature table, features were built before cutoff, the Phase 5 model was trained on the leakage-safe Phase 4 training dataset, and Phase 6 does not use labels or target-window events.

Privacy checks: Scoring artifacts are designed as aggregate summaries only. Row-level score data is written under ignored local processed data and should not be committed.

Current result: The Phase 6 batch scoring job completed successfully. It generated one score row per eligible client and produced aggregate scoring artifacts suitable for review.

Next step: The score table has already been used for local API lookup serving. Remaining review points are score refresh cadence, threshold choice, and whether the 500000-row local serving export is sufficient for API/cache testing.

API Serving Milestone Summary

Objective: Create a lightweight lookup API for purchase propensity scores generated by the batch scoring pipeline.

Serving architecture: The API serves exported batch scores from SQLite. It does not load Spark or run Spark ML model inference per request.

Input scoring table: `data/processed/scoring/purchase_propensity_scores/`

Serving export format: SQLite lookup database at `data/serving/purchase_propensity_scores.sqlite`.

API endpoints:

- `GET /health`
- `GET /metadata`
- `GET /scores/{client_id}`
- `POST /predict`

Response fields:

- `client_id`
- `prediction_score`
- `prediction_label`
- `model_version`
- `scored_at`

Manual prediction response fields:

- `prediction_score`
- `prediction_label`
- `decision`
- `model_version`
- `missing_features_filled`
- `used_feature_count`

Testing strategy: API and repository tests use a temporary SQLite database, fake client IDs, and fake model metadata only. Tests cover health, metadata, successful score lookup, missing score lookup, missing database handling, manual prediction success, missing metadata handling, invalid manual feature values, score bounds, and binary prediction labels.

Privacy and security checks: API examples use fake client IDs and fake manual feature examples only. The generated SQLite database and local model metadata are ignored by git. The API does not expose raw labels, query text, product names, model binaries, or row-level examples from real data.

Current result: Phase 7A lookup API code, SQLite repository, serving export job, fake-data tests, sanitized serving artifacts, and the 500000-row local SQLite demo export are implemented. Phase 7B manual feature-input prediction is implemented through lightweight exported Logistic Regression metadata and `POST /predict`. The Windows runtime export failed while Spark was reading the local Parquet score output, so WSL/Linux or a properly configured Spark runtime should be used to refresh the serving DB.

Limitations: This is a local MVP serving layer. It does not include external database deployment, authentication, Redis/distributed cache, production monitoring, or load testing. Manual prediction is useful for review, but it depends on keeping exported metadata synchronized with the trained model version.

Next step: Review the lookup and manual prediction API behavior before Phase 7C demo UI work.

Updated Serving and Demo Plan

Objective: Clarify the remaining product/demo phases so the API, UI, and model tuning work stay lightweight, reviewable, and separated from heavy Spark processing.

Batch score lookup: The API serves `client_id` lookup requests from the exported batch score store. This keeps request handling fast and avoids reading raw data, loading Spark, or running Spark model inference for each request.

Manual feature-input prediction: Phase 7B adds manual feature entry for review and what-if checks through `POST /predict`. This path uses exported lightweight Logistic Regression metadata: feature order, imputation values, coefficients, intercept, and threshold. The API computes the logistic sigmoid score directly from those values without Spark in the request path.

Offline training and tuning: Training, model comparison, and hyperparameter tuning should remain offline Spark jobs. The frontend should not trigger heavy Spark training by default, and the API should not train models inside request handlers.

Demo interface: A simple Streamlit app or lightweight frontend is still planned for mentor review because it can show both serving modes: batch score lookup and manual feature-input prediction. Any examples should use fake identifiers and neutral values only.

Experiment comparison: Model variants should be trained offline and recorded as sanitized aggregate artifacts under `artifacts/experiments/`. The comparison view should summarize ROC-AUC, PR-AUC, TopK precision/recall/lift, confusion matrix, selected threshold, and parameter set for each variant. It should not store or display raw predictions, real client IDs, or row-level examples.

Privacy and PII Guardrails

The project treats `client_id` as a linkable pseudonymous user identifier. Even when it is not a name or email address, it can still connect behavior across events, features, labels, scores, and serving outputs. Prediction scores are also sensitive because they summarize user-level behavior and likelihood to buy.

Guardrails used throughout the MVP:

- Raw row-level samples are not written to shared artifacts.
- Raw `client_id` values are not included in reports or committed artifacts.
- Raw search query text is not written to shared artifacts.
- Product names are not written to shared artifacts.
- Row-level prediction examples with real users are not included.
- Processed data, scoring outputs, SQLite serving DBs, local model metadata, and model binaries remain local and ignored by git.
- API and repository tests use fake client IDs and fake model metadata.
- Reports and artifacts use aggregate metrics, schema fields, validation counts, and sanitized technical notes only.

Reason: The goal is to make the work mentor-reviewable without exposing user-level behavioral data. This is especially important for scoring projects because both identifiers and prediction outputs can be sensitive. Before any production deployment, the API would need access control, authentication, logging review, and clear policy around who can request or view user-level scores.

9. Risks and Open Questions

- Demo UI is not implemented yet.
- Load/stress testing is not implemented yet.
- External database deployment is not implemented.
- Authentication, access control, rate limiting, and production monitoring are not implemented.
- Redis or distributed cache is not implemented.
- Time-based backtesting over multiple cutoffs has not been run.
- Threshold selection remains a review point because the default threshold produces high recall and many false positives.
- Duplicate handling policy remains a review point because processed duplicate events may affect feature magnitudes.
- `page_visit` remains deferred for MVP features because it is the largest table and was not needed for the initial serving path.
- Windows local Spark had filesystem/native Hadoop issues for some Parquet operations; WSL/Linux or a properly configured Spark runtime is needed for reliable refresh jobs.

10. Current Milestone

Current milestone: Phase 7: API Serving & Demo Interface.

Done in this milestone:

- serving export job added
- FastAPI lookup app added
- SQLite repository added
- health, metadata, and score endpoints added
- manual prediction endpoint added
- lightweight model metadata export job added
- pure-Python Logistic Regression scoring logic added
- fake-data API tests added
- local model metadata export completed with 36 features
- 500000-row API/cache serving export completed locally
- sanitized API contract and serving validation artifacts added
- README, jobs README, report, and PLAN updated for Phase 7A/7B

Not included yet:

- demo UI
- offline model variant comparison
- load or stress testing
- external database deployment
- authentication
- production deployment

11. Next Steps

1. Review lookup and manual prediction API behavior.
2. Keep `data/serving/`, SQLite files, and local model metadata excluded from git.
3. Decide whether the demo UI should use Streamlit or a lightweight frontend.
4. Run API/cache load testing after the demo scope is approved.
5. Keep offline tuning and experiment comparison in Phase 8, separate from API request handling.